



STAGE SGSI

-

CRÉATION D'UN CATALOGUE DES APIs

Documentation technique

Auteur :

KHEIRALLAH Cédric

Superviseur :

MINNE Frédéric

Dernière révision :

28 août 2026

Table des matières

1	Vu d'ensemble du projet	1
1.1	Fonctionnalités clés	1
1.1.1	Fonctionnalités utilisateurs	1
1.1.2	Fonctionnalités administrateurs	1
2	Architecture de l'application	2
2.1	Vue globale	2
2.2	Services	2
2.3	Communication entre les services	3
3	Organisation du projet	4
4	Service backend (FastAPI)	5
4.1	backend.py	5
4.2	ImportationManager	5
4.3	ApiArchiveExtractor	5
4.4	ApiManagerClientInterface et ApiCtlClient	6
4.4.1	ApiManagerClientInterface	6
4.4.2	ApiCtlClient	6
4.5	utils.py	7
5	Service frontend (Flask)	7
5.1	frontend.py	7
5.2	api_metadata_extractor.py	8
6	Service scheduler (Supercronic)	8
7	Conteneurisation et déploiement	8
7.1	Image backend	8
7.2	Image frontend	9
7.3	Image scheduler	9
7.4	Configuration des services	9
7.4.1	Réseaux	9
7.4.2	Volumes et fichiers de configuration	9
7.4.3	Variables d'environnement	10
8	Notes, limitations et améliorations	10
8.1	Limitations connues	10
8.1.1	Redoc et Bootstrap 5+	10
8.1.2	Disparition temporaire des APIs sur l'index lors du rafraîchissement	10
8.2	Objectifs futurs et améliorations	10

1 Vu d'ensemble du projet

API Catalog est une application web permettant de parcourir les APIs publiées sur une instance WSO2 API Manager, d'afficher leur documentation et de la régénérer automatiquement de manière régulière. Son usage est destiné aux informaticiens du SGSI, des utilisateurs et/ou consultants externes de l'UCLouvain qui souhaitent consulter les APIs publiées sur l'instance API Manager du SGSI afin de faire savoir leur intérêt d'avoir accès à des APIs particulières.

Dans ce document nous décrirons l'architecture, la structure, le fonctionnement et les limitations de l'application. Nous terminerons par des points d'amélioration et une conclusion. Pour plus de détails concernant des méthodes privées particulières, le code est documentée avec des docstrings et des commentaires pour expliquer certains choix spécifiques mais trop détaillés pour être inclus ici où nous resterons sur une description "dans les grandes lignes".

1.1 Fonctionnalités clés

1.1.1 Fonctionnalités utilisateurs

- Consulter la liste de toutes les APIs publiées sur l'instance API Manager du SGSI (à l'exception des APIs blacklistées par les administrateurs)
- Consulter les spécifications OpenAPI de chaque API (`swagger.yaml`) ainsi que les documents potentiellement associés
- Télécharger la documentation de chaque API ainsi que les documents potentiellement associés

1.1.2 Fonctionnalités administrateurs

- Consulter et modifier le répertoire de stockage partagé des APIs actuellement présents sur le catalogue
- Rafraîchir les données des APIs sur le catalogue manuellement via une requête HTTP
- Blacklister des APIs afin de les exclure du répertoire de stockage partagé (et donc également de l'affichage sur le catalogue) lors le prochain rafraîchissement
- Ajuster la fréquence de rafraîchissement automatique du catalogue

2 Architecture de l'application

2.1 Vue globale

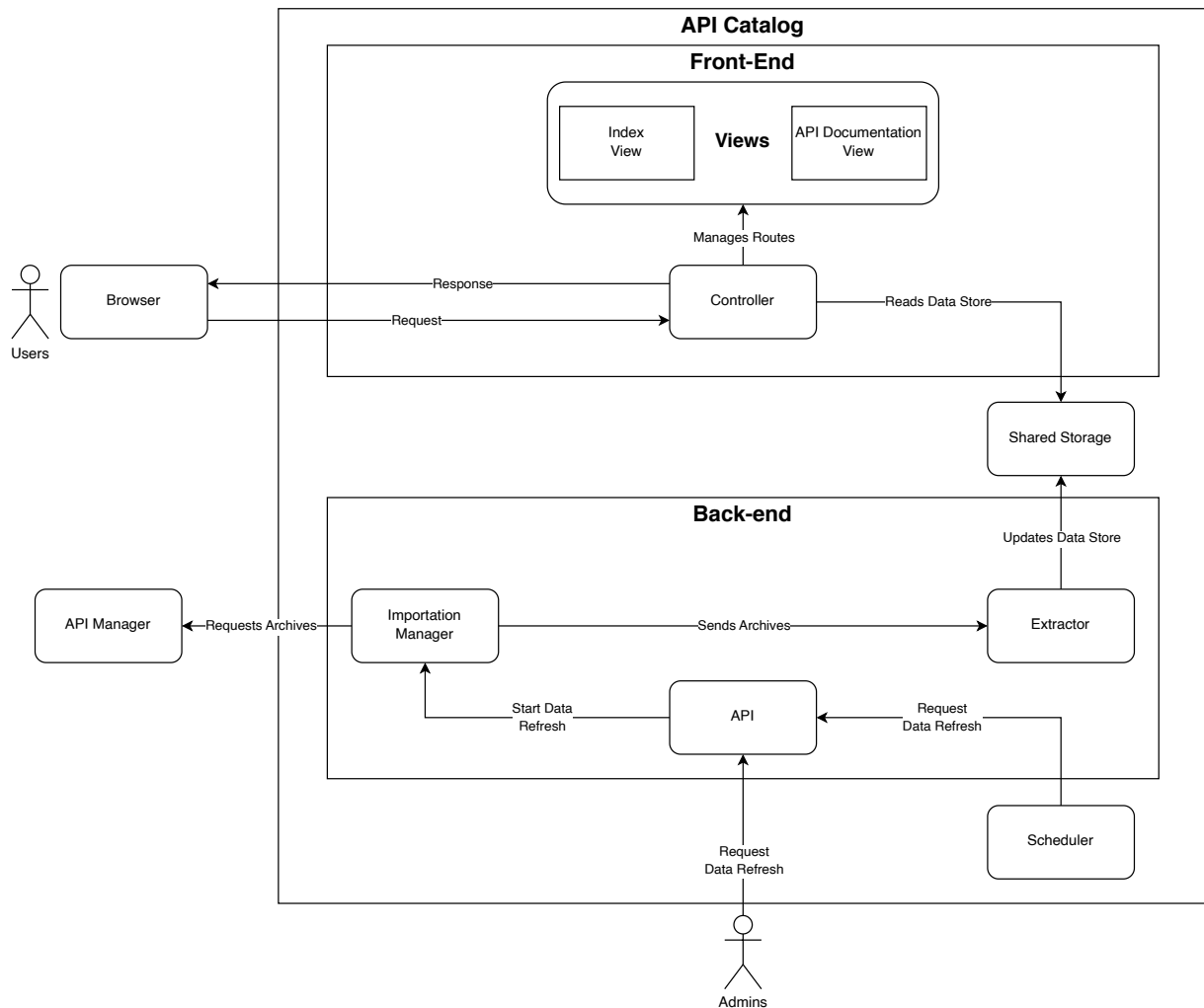


FIGURE 1 – Diagramme d'architecture

2.2 Services

Le catalogue est déployée en conteneurs via Docker Compose et est constituée de trois services :

- **backend** : une application FastAPI recevant des requêtes afin d'importer les APIs publiées sur l'API Manager (via l'outil en ligne de commande fourni par WSO2 : apictl) et de les extraire dans un répertoire de données partagé.
- **frontend** : une application Flask chargée d'afficher le catalogue et la documentation de chaque API à partir du répertoire de données partagé.
- **scheduler** : un lanceur de tâches automatique Supercronic lançant automatique le rafraîchissement automatique en appelant le backend à horaire régulier.

2.3 Communication entre les services

Les trois services **backend**, **frontend** et **scheduler** communiquent ensemble via le réseau interne de Docker (**bridge**) et ne sont pas exposés à l'extérieur sauf par l'hôte qui a également accès à ce réseau. Le backend et frontend sont également connectés au proxy Nginx. Le serveur de production n'étant pas disponible pour ce projet, une configuration supplémentaire est fournie afin de déployer un proxy avec des certificats auto-signés pour faciliter le développement. Plus de détails dans la section *Conteneurisation et déploiement*⁷.

3 Organisation du projet

```
1 apictl/
2     login.sh                                # generates login key from credentials
3     ...
4 backend/
5     backend.py
6     importation_manager.py
7     api_archive_extractor.py
8     apictl_client.py
9     api_manager_client_interface.py
10    utils.py
11    blacklist.txt                          # API blacklisting file
12    requirements_backend.txt               # Python dependencies
13    uvicorn_logging.json                  # Uvicorn logging configuration
14 data/
15 docker/
16     Dockerfile.backend
17     Dockerfile.frontend
18     Dockerfile.scheduler
19 frontend/
20     static/
21         font/Montserrat.woff2            # web font
22         icon/...                          # static icons (PNG, SVG)
23         redoc/
24             redoc.standalone.js          # Redoc standalone script
25         ...
26         filter.js                        # sorting and filtering logic
27         style.css                        # frontend styling
28     templates/
29         403.html                         # HTTP 403 error page
30         404.html                         # HTTP 404 error page
31         415.html                         # HTTP 415 error page
32         base.html                        # base for all pages (navbar + footer)
33         card.html                        # card template for index page
34         contact.html                     # static contact page
35         documentation.html                # documentation page for an API
36         footer.html                      # global page footer
37         index.html                       # index page listing all APIs
38         navbar.html                      # global page navigation bar
39         redoc.html                       # Redoc template for Swagger rendering
40         viewer.html                      # standalone text/md/html viewer
41     frontend.py
42     api_metadata_extractor.py
43     requirements_frontend.txt             # Python dependencies
44     gunicorn_logging.json                 # Gunicorn logging configuration
45 proxy/
46     deploy/
47         certs/...                        # self-signed certificates
48         proxy.conf                       # proxy configuration file
49         snakeoil.sh                      # generates self-signed certificates
50 scheduler/crontab                       # task frequency configuration
51 compose.yaml
52 compose.proxy.yaml
53 .editorconfig                           # editor style auto-configuration
54 .env.example                            # example environment variables
55 ...
```

Listing 1 – Structure du projet

4 Service backend (FastAPI)

4.1 backend.py

Le backend FastAPI est le point d'entrée du backend. C'est ici que les requêtes HTTP pour rafraîchir les APIs sont reçues et traitées. Il est responsable de :

- Charge la configuration via des variables d'environnement (`.env`), notamment `DEBUG` (niveau de log), `APIM_ENVIRONMENT` (environnement API Manager) et `SERVER_URL` (URL de base utilisée pour réécrire les serveurs dans les fichiers Swagger)
- Initialise les répertoires `tmp/` (à priori existant dans le conteneur donc redondant) et `data/` au même niveau que `backend/`
- Instancie le logger et les classes : `ApiCtlClient`, `ApiArchiveExtractor` et `ImportationManager`
- Expose deux routes pour rafraîchir les données d'APIs :
 - `POST /refresh` : rafraîchit toutes les APIs publiées
 - `POST /refresh/{api_type}/{api_name}-{api_version}` : rafraîchit une API spécifique via son type, son nom et sa version
 - Gestionnaire d'erreurs FastAPI pour retourner des exceptions HTTP appropriés selon les réponses de l'ImportationManager ou de l'entrée utilisateur (400 Bad Request, 404 Not Found, 500 Internal Server Error)

4.2 ImportationManager

L'ImportationManager est la "super classe" qui organise l'ensemble du processus de rafraîchissement en coordonnant le client API Manager et `ApiArchiveExtractor`.

- `refresh_all()` : rafraîchit toutes les APIs publiées et retourne la liste des APIs traitées
 1. Purge intégralement `data/` (à l'exception de `.gitkeep` qui permet de conserver un répertoire vide dans git)
 2. Récupère la liste des APIs actuellement publiées sur API Manager
 3. Importe et extrait chaque API une à une
 4. Retourne la liste des APIs traitées (une entrée `None` signalant une API blacklistée)
- `refresh_single(api_type, api_name, api_version)` : rafraîchit une seule API à la demande par son type, nom et version et retourne le nom et version de l'API si le rafraîchissement a réussi ou `None` si l'API est blacklistée
 1. Vérifie si l'API est blacklistée (et retourne `None` le cas échéant)
 2. Crée le sous-répertoire temporaire nécessaire
 3. Importe l'archive depuis API Manager vers `tmp/`
 4. Extrait l'archive vers `data/`
 5. Purge `tmp/` dans un bloc `finally` que l'opération réussisse ou échoue (cela évite l'accumulation d'archives dans un cas où l'importation réussit mais l'extraction échoue)

4.3 ApiArchiveExtractor

L'ApiArchiveExtractor est responsable de l'extraction et de la normalisation des archives `.zip` exportées par `apictl` depuis API Manager. Il est appelé par l'ImportationManager pour traiter les archives après leur importation.

- `extract_api_archive(api_type, api_name, api_version)` : extrait l'archive de l'API dans `tmp/` dans `data/` en filtrant et réécrivant certains fichiers selon le type d'API (API ou API product)
 1. Localise l'archive dans `tmp/apis/` ou `tmp/api-products/`
 2. Extrait le contenu dans `data/` en excluant certains fichiers selon le type (`api_meta*` pour les APIs, `api_product_meta*` et `APIs/*` pour les API products)
 3. Renomme `api_product.yaml` en `api.yaml` pour les API products, afin d'uniformiser le format entre les deux types d'APIs (on ne fait pas leur différence dans le catalogue mais API Manager crée des structures assez différentes selon que l'API est un produit ou non)
 4. Filtre les métadonnées de `api.yaml` pour ne conserver que les champs utiles (`name`, `provider`, `context`, `version`, `tags`, `lastUpdatedTime`)
 5. Réécrit l'URL du serveur dans `Definitions/swagger.yaml` en combinant `SERVER_URL` avec le contexte et la version de l'API
 6. Ajoute un suffixe aux fichiers de documentation inline dans `Docs/` (`INLINE` → `.html`, `MARKDOWN` → `.md`) afin de permettre leur visualisation directe dans le catalogue

4.4 ApiManagerClientInterface et ApiCtlClient

`ApiManagerClientInterface` est une interface abstraite définissant les méthodes qu'un client d'API Manager devrait implémenter et fournir. Elle est utilisée par l'`ImportationManager` pour interagir avec API Manager.

La classe qui l'implémente actuellement est `ApiCtlClient`, un outil en ligne de commande permettant d'interagir avec API Manager mais celui-ci pourrait aisément être remplacé par un autre outil (ce sera d'ailleurs le cas avec Admin API v4) sans avoir à modifier le reste de l'application puisque c'est bien l'interface qui est appelée par l'`ImportationManager`.

4.4.1 ApiManagerClientInterface

Une interface définissant le contrat que doit respecter tout client d'interaction avec API Manager :

- `get_published_apis()` : liste toutes les APIs publiées (nom, version, type)
- `import_single_archive(api_type, api_name, api_version)` : importe une archive unique et retourne son chemin

4.4.2 ApiCtlClient

Implémente l'interface ci-dessus en s'appuyant sur l'outil en ligne de commande de WSO2 `apictl` (API Controller) :

- `get_published_apis()` : liste toutes les APIs publiées (nom, version, type)
 1. Exécute `apictl get apis/apictl get api-products` en filtrant les APIs publiées
 2. Reconstitue et parse la sortie JSON ligne par ligne
 3. Retourne une liste de dictionnaires avec `name`, `version` et `type` pour chaque API
- `import_single_archive(api_type, api_name, api_version)` : importe une archive unique et retourne son chemin
 1. Vérifie d'abord que l'API demandée existe et est bien publiée (vérification si il s'agit d'un rafraîchissement ciblé)

2. Exécute ensuite `apictl export` pour générer l'archive
3. Reconstitue le chemin d'export attendu (`~/wso2apictl/exported/...`) où le répertoire `.wso2apictl` est toujours créé dans le répertoire racine `$HOME` du système y compris en conteneur ou cela se trouve dans `/root`

4.5 `utils.py`

Un simple module utilitaire permettant de faire des opérations sur les fichiers et répertoires ainsi que lire et écrire des fichiers YAML. Il est utilisé à plusieurs endroits du backend pour éviter la duplication de code.

- `copy()` / `move()` : opérations sur les fichiers et répertoires
- `purge_directory()` : vide un répertoire en excluant certains motifs (par exemple `.git*` pour ne pas supprimer tous les fichiers commençant par `.git`)
- `yaml_to_dict()` / `dict_to_yaml()` : lecture et écriture de fichiers YAML

5 Service frontend (Flask)

5.1 `frontend.py`

Le frontend Flask est le point d'entrée du frontend. C'est ici que les requêtes HTTP pour afficher le catalogue et la documentation des APIs sont reçues, traitées et l'utilisateur servi selon sa requête.

- Charge la configuration via des variables d'environnement (`.env`), notamment `DEBUG` (niveau de log)
- Initialise le répertoire `data/` au même niveau que `frontend/`
- Instancie le logger
- `validate_api_file_path(api_dir_name, relative_path)` : vérifie que le chemin d'accès à un fichier de l'API est valide et ne permet pas d'accéder à des fichiers en dehors du répertoire de l'API. Cela évite que les utilisateurs ne puisse aller lire l'entiereté du système de fichiers du conteneur via des chemins relatifs (ex : `../../../../etc/...`)
- Expose plusieurs routes pour afficher l'index du catalogue et la documentation des APIs ainsi que les documents associés :
 - `GET /` : redirige vers `/index`
 - `GET /index` : affiche la page d'accueil listant toutes les APIs (triées par nom)
 - `GET /contact` : affiche la page de contact statique
 - `GET /documentation/<api_name>--<api_version>/` : affiche la page de documentation d'une API (Swagger + documents associés)
 - `GET /documentation/<api_dir_name>/<relative_path>` : sert un fichier statique de l'API (document, swagger, icône)
 - `GET /documentation/inline/<api_dir_name>/<relative_path>` : affiche un rendu pour les formats supportés (`.md/.html/.txt`)
 - Gestionnaire d'erreurs Flask pour afficher des pages d'erreur personnalisées pour 403 Forbidden, 404 Not Found et 415 Unsupported Media Type

5.2 `api_metadata_extractor.py`

Un module utilitaire qui permet d'extraire les métadonnées utiles des répertoires de chaque API afin de les passer au frontend pour utilisation.

- `get_api_metadata(api_dir, logger)` : lit le répertoire de l'API spécifié et retourne un dictionnaire contenant ses métadonnées :
 - `name` : nom de l'API
 - `version` : version de l'API
 - `context` : contexte de l'API
 - `provider` : fournisseur de l'API
 - `tags` : tags associées à l'API (actuellement obsolète car inutilisé dans l'instance API Manager de production du SGSI)
 - `last_updated` : date de la dernière mise à jour
 - `icon_url` : URL de l'icône de l'API
 - `fallback_bg` : couleur de fond de substitution
 - `swagger_url` : URL du fichier Swagger
 - `documents` : documents associés à l'API

6 Service scheduler (Supercronic)

Supercronic est un outil permettant de planifier l'exécution automatique de tâches spécifiquement développé afin de tourner en conteneur. Il est basé sur cron, le même système présent sur les systèmes Linux ("`man cron`" dans un terminal pour en savoir plus sur l'outil). Il est utilisé dans ce projet pour déclencher automatiquement le rafraîchissement des APIs à intervalle régulier.

- Sa configuration se fait via le fichier `scheduler/crontab` synchronisé en temps que fichier de configuration dans Docker Compose
- Par défaut, un rafraîchissement complet du catalogue est déclenché toutes les heures (à la minute 0) en appelant la route `POST /refresh` du backend
- Les logs de debug (affichant la prochaine exécution planifiée) sont activés par défaut, ils peuvent être désactivés en dé-commentant la commande sans le flag de debug dans `compose.yaml`

7 Conteneurisation et déploiement

L'application est conteneurisée et déployée via Docker Compose. `compose.yaml` définit les trois services principaux (**backend**, **frontend**, **scheduler**) et leurs configurations. Un fichier optionnel `compose.proxy.yaml` permet de déployer avec un service proxy Nginx utilisant des certificats auto-signés pour faciliter le développement et les tests (un script `proxy/snakeoil.sh` est fourni pour générer ces certificats).

7.1 Image backend

Basée sur une image `python:3.14-slim` (évite l'installation manuelle de Python dans l'image)

1. Installe et initialise `apictl` (attention que les clés de connexion doivent avoir été générées avant avec le script `apictl/login.sh`)
2. Copie les fichiers sources du backend et installe les dépendances Python nécessaires
3. Définit le point d'entrée et la commande de lancement du backend FastAPI via Uvicorn

7.2 Image frontend

Basée sur une image `python:3.14-slim` (évite l'installation manuelle de Python dans l'image)

1. Copie les fichiers sources du frontend et installe les dépendances Python nécessaires
2. Définit le point d'entrée et la commande de lancement du frontend Flask via Gunicorn

7.3 Image scheduler

Basée sur une image `alpine:3.21`.

- Télécharge et installe le binaire Supercronic
- Copie le fichier de configuration `crontab` dans l'image
- Définit le point d'entrée et la commande de lancement de Supercronic avec le fichier de configuration

7.4 Configuration des services

7.4.1 Réseaux

Deux réseaux Docker sont utilisés pour permettre la communication entre les services ainsi qu'avec l'extérieur :

- **internal** : réseau interne de type **bridge** utilisé pour la communication entre les services backend, frontend et scheduler. Il n'est pas exposé à l'extérieur
- **nginx-proxy** : réseau externe utilisé pour la communication entre les services backend, frontend et le proxy Nginx, le service scheduler n'a pas besoin d'y être connecté puisqu'il ne doit pas y être accédé depuis l'extérieur

7.4.2 Volumes et fichiers de configuration

- **data/** : **volume** partagé entre le backend, le frontend et l'hôte pour stocker les données des APIs. Le backend y écrit les nouvelles données d'API, le frontend les lit (lecture seule du côté frontend) et l'hôte peut y lire et écrire les données sans avoir à redéployer les services.
- **blacklist.txt** : **fichier de configuration** partagé entre le backend et l'hôte pour stocker la liste des APIs blacklistées. Le backend y lit les APIs à exclure lors du rafraîchissement et l'hôte peut y écrire pour ajouter ou retirer des APIs de la liste sans avoir à redéployer le service.
- **crontab** : **fichier de configuration** partagé entre le scheduler et l'hôte pour définir la fréquence des rafraîchissements automatiques. Le scheduler y lit la configuration et l'hôte peut y écrire pour modifier la planification sans avoir à redéployer le service.

- `docker.sock` : **volume** partagé entre le proxy Nginx et l'hôte pour permettre au proxy de détecter automatiquement les services et de configurer les hôtes virtuels. Il est utilisé par le proxy pour intercepter les requêtes HTTP et les rediriger vers les services backend et frontend appropriés. Uniquement présent lorsque le proxy de développement est déployé via `compose.proxy.yaml`.
- `proxy.conf` : **volume** partagé entre le proxy Nginx et l'hôte pour paramétrer le proxy. Uniquement présent lorsque le proxy de développement est déployé via `compose.proxy.yaml`.
- `certs/` : **volume** partagé entre le proxy Nginx et l'hôte pour stocker les certificats TLS utilisés par le proxy. Uniquement présent lorsque le proxy de développement est déployé via `compose.proxy.yaml`.

7.4.3 Variables d'environnement

Docker compose utilise des variables d'environnement supplémentaires que celles déjà définies dans le fichier `.env` pour configurer les services :

- `VIRTUAL_PORT` : port interne sur lequel le service écoute (5000 pour Flask et 8000 pour FastAPI)
- `VIRTUAL_HOST` : nom d'hôte associé au service et utilisé par le proxy pour déterminer le routage des requêtes
- `TZ` : fuseau horaire utilisé par les services

8 Notes, limitations et améliorations

8.1 Limitations connues

8.1.1 Redoc et Bootstrap 5+

Les spécifications OpenAPI des APIs sont affichées via Redoc qui offre un rendu très complet mais qui semble avoir des problèmes de compatibilité avec Bootstrap 5+ (qui permet de styliser les pages web sans avoir besoin d'un CSS complexe) où la barre de navigation sur la gauche ne réagit plus du tout. Une des solutions serait apparemment de passer à une version antérieure de Bootstrap (non-testé et pourrait casser si Bootstrap est mis à jour) ou d'override le CSS de Redoc (non-testé et pourrait casser si Redoc est mis à jour).

8.1.2 Disparition temporaire des APIs sur l'index lors du rafraîchissement

Le rafraîchissement complet du catalogue peut prendre un moment (surtout en production où il y a beaucoup d'APIs publiées) et la majorité du temps de rafraîchissement est dû à l'importation des archives depuis API Manager alors que l'extraction des archives est très rapide. Cela signifie que pendant quelques secondes le catalogue apparaît incomplet à un utilisateur qui consulterait le catalogue à ce moment là. Une idée pour contourner ce problème serait de garder en cache les anciennes données et ne remplacer le répertoire de données que lorsque le rafraîchissement est terminé (en extrayant d'abord les nouvelles données dans `tmp/` par exemple).

8.2 Objectifs futurs et améliorations

Le fichier `TODO.md` contient une liste de points d'amélioration et de bugs à fixer.